

Hardware/Software Co-design: Partitioning

System Level Design and Modeling

Preeti Ranjan Panda

Department of Computer Science and Engineering

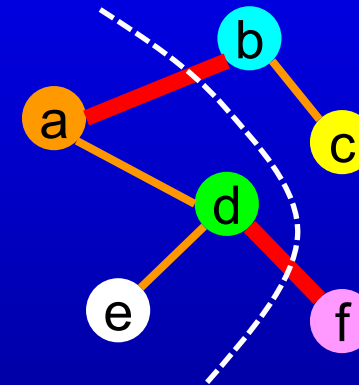
Indian Institute of Technology Delhi

System Partitioning

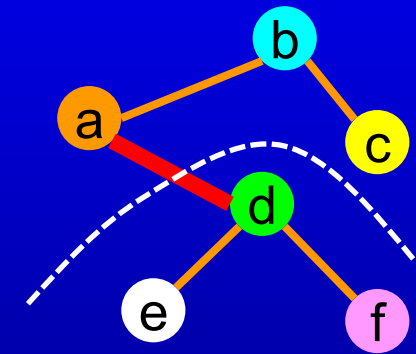
- Given a high-level specification consisting of Modules
 - Which modules should be implemented in Hardware?
 - Which modules should be implemented in Hardware?
- Related problem: Given a network modelled as graph
 - nodes = computation modules
 - edges = communication channels
 - Divide the nodes into TWO PARTITIONS
 - Minimise #edges crossing partitions
- Several applications of problem variations
 - Circuits fitting into chips
 - Hardware vs. Software partitioning

Partitioning Heuristics: Kernighan-Lin Algorithm

- Problem: Partition graph into 2 equal sets of nodes
 - minimum #edges (or sum of edge weights) crossing the partitions
 - Heuristic: optimal is hard



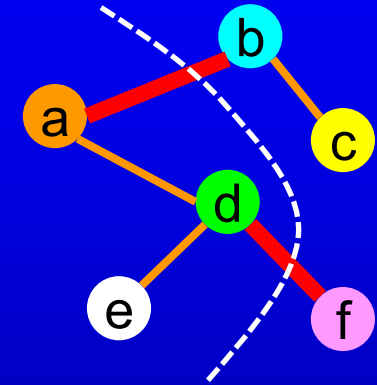
Cost = 2



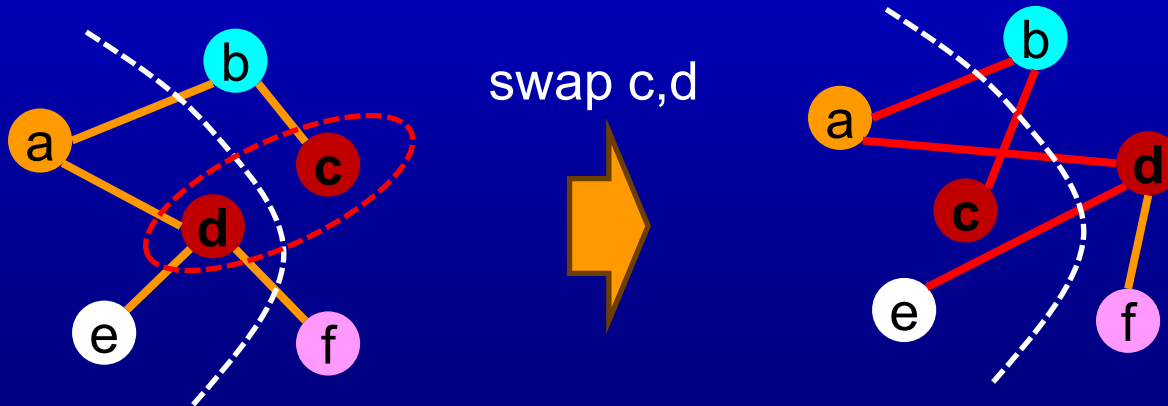
Cost = 1
Optimal

Partitioning Heuristics: Kernighan-Lin Algorithm

- Idea:
 - Start with any random partition (equal size)
 - Select pair of nodes, one from each partition
 - Exchange if lower cost
 - Iterate until no more improvement

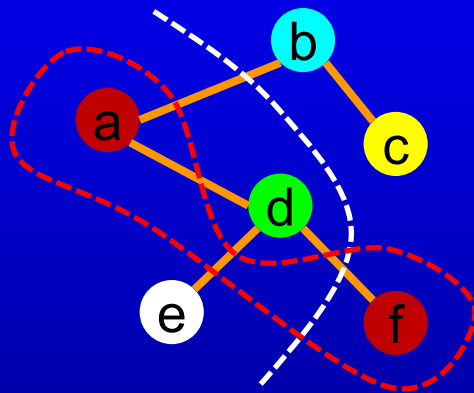


Random Initial: Cost = 2



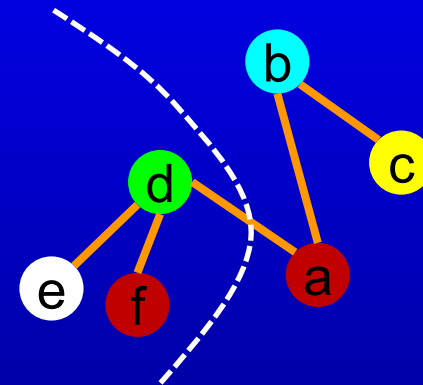
Cost after swap = 4 (worse)

Partitioning Heuristics: Kernighan-Lin Algorithm



Cost = 2

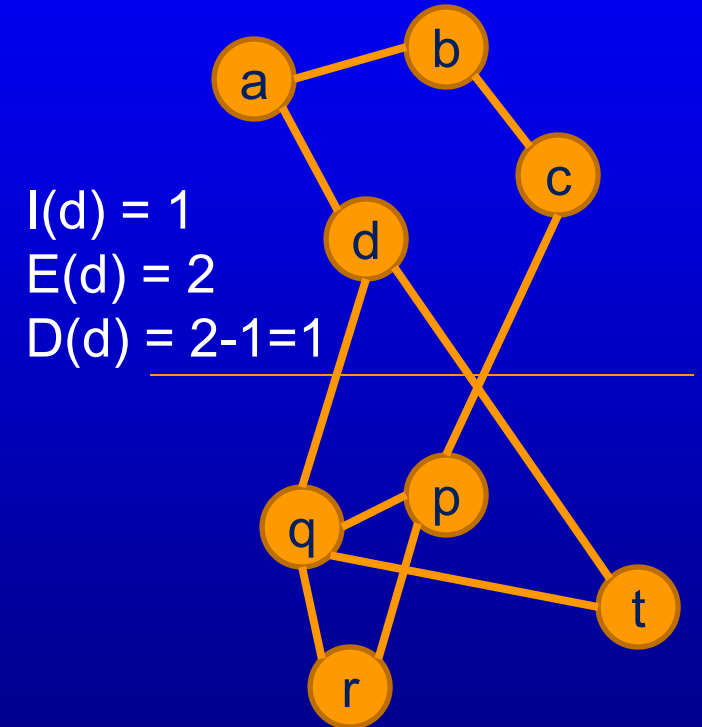
swap a,f



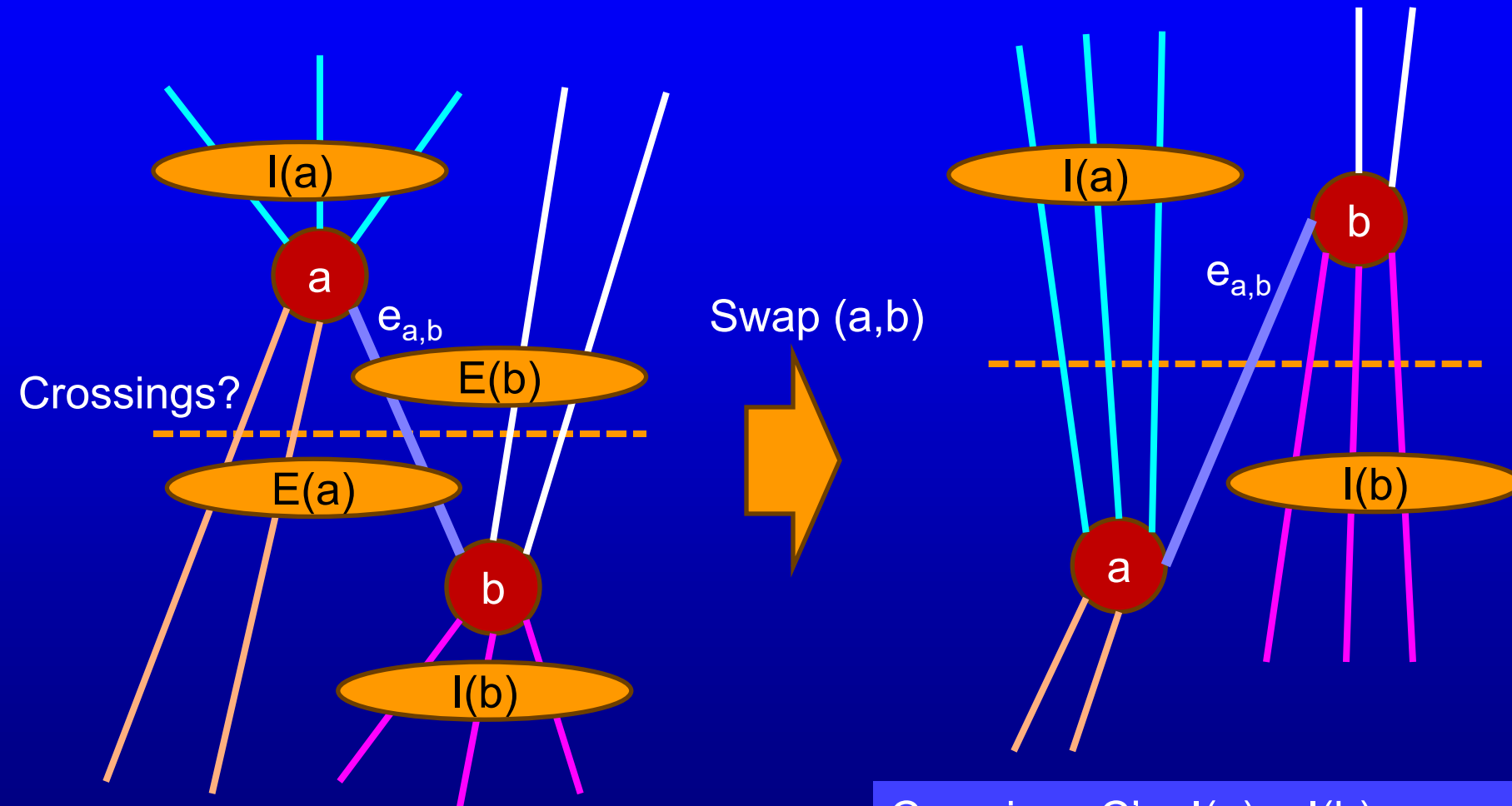
Cost after swap = 1 (better)

Cost Computation

- **Internal cost $I(a)$**
 - sum of connected edge weights to nodes in same partition
- **External cost $E(a)$**
 - sum of connected edge weights to nodes in other partition
- **Cost Difference $D(a) = E(a) - I(a)$**
 - change in costs if 'a' is moved to other partition
 - how many fewer crossings?
- **Change in cost (= $\text{gain}(a,b)$) if we exchange a,b?**



How many crossings reduce on Swap (a,b)?



$$\text{Crossings } C = E(a) + E(b) - e_{a,b}$$

$$\text{Crossings } C' = I(a) + I(b) + e_{a,b}$$

$$\begin{aligned} \text{Reduction in Crossings} \\ \text{gain (a,b)} &= C - C' \\ &= E(a) + E(b) - e_{a,b} \\ &\quad - [I(a) + I(b) + e_{a,b}] \\ &= [E(a) - I(a)] \\ &\quad + [E(b) - I(b)] - 2e_{a,b} \\ &= D(a) + D(b) - 2e_{a,b} \end{aligned}$$

Algorithm

- for $i = 1$ to $n/2$
 - identify $(x[i], y[i])$ for which $\text{gain}(x[i], y[i])$ is maximum
 - note and remove pair
 - update costs of remaining nodes
- How does cumulative gain (sum of gains so far) change with i ?
- Exchange $x[1..k]$ and $y[1..k]$ for k that maximises cumulative gain. Back to step 1.
- If no gain, stop

K-L Algorithm

- Small number of outer loop iterations in practical examples
- Equal partitions
- No node weights
- Time Complexity...

Complexity Analysis

- Computing D values?
- Each swap requires updating D values
 - how many updates?
- Total time updating D values?
- Selecting the best pair?
- Complexity for each pass?
- Overall complexity for d passes?

Complexity Analysis

- Computing D values? $O(n^2)$
- Each swap requires updating D values
 - how many updates? (at i-th step, up to $2(n-i)$ updates)
- Total time updating D values? $O(n^2)$
- Selecting the best pair? $O((n-i+1)^2)$
- Complexity for each pass? $O(n^3)$
- Overall complexity for d passes? $O(dn^3)$

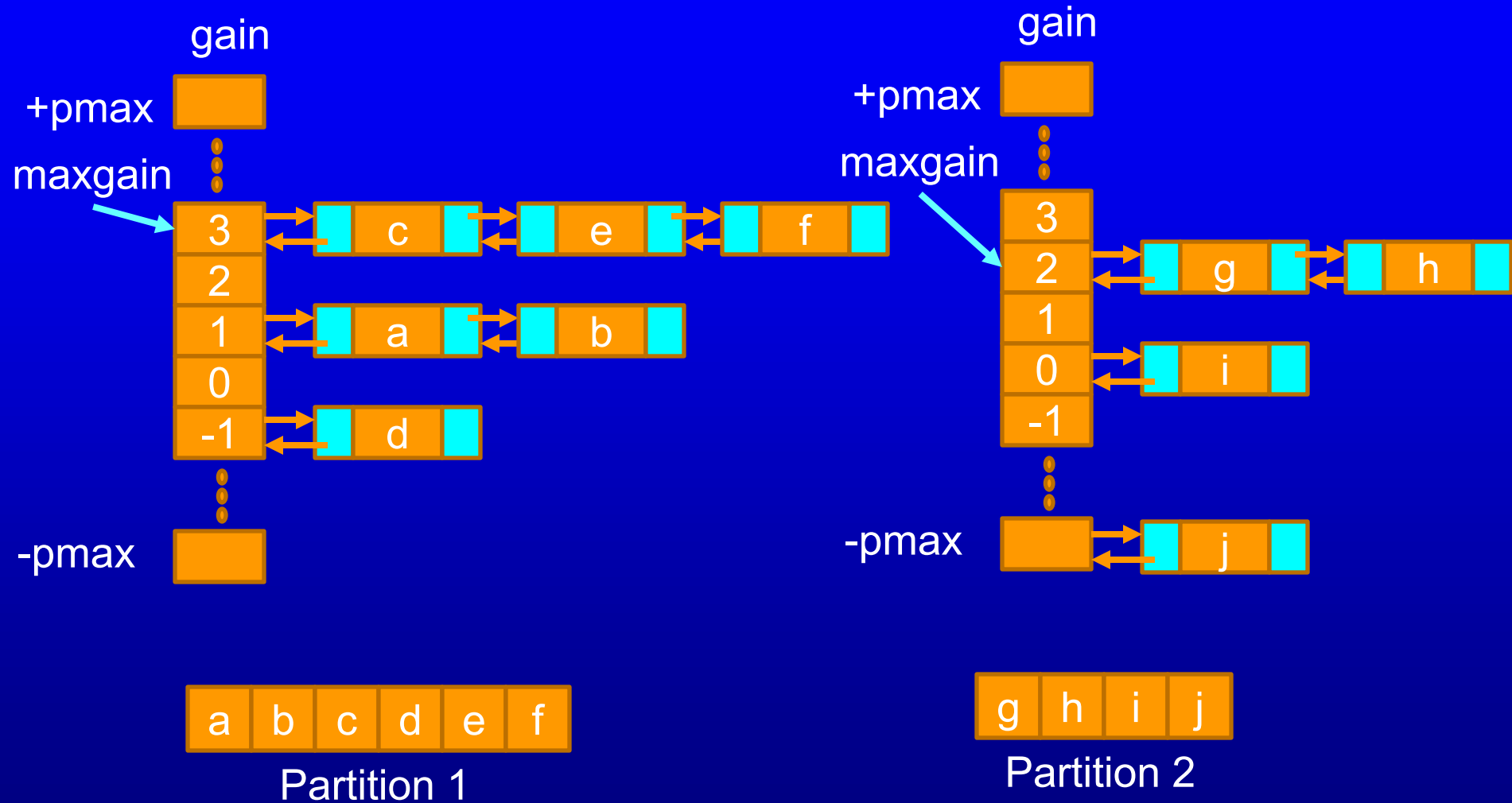
Fiduccia-Mattheyses Algorithm

- Alternative partitioning heuristic
- Extension to Hypergraph
 - an edge can connect multiple nodes
 - more accurate representation of wires
- Extension to arbitrary ratios of partition weights
 - not necessarily equal partitions
 - nodes can have weights

Variation: Move Nodes

- Move nodes instead of edges
- Maintain BALANCE condition
 - ratio of node weight sums
- Overall strategy similar to K-L
 - sequence of tentative node moves
 - when no more improvement, move the set of nodes maximising cumulative gain
 - repeat

Data Structure Improvement



Time Complexity

- Identifying node with max gain? $O(1)$
- Moving the node to other partition? $O(1)$
- Enumerating neighbours? $O(e)$
 - Updating gain for each neighbour? $O(1)$
 - Moving neighbour to appropriate bucket? $O(1)$
- Complexity for each pass? $O(e)$
- Overall Complexity for d passes? $O(de)$

Non-deterministic Partitioning Algorithms

- K-L and F-M are Deterministic
 - predictable complexity
 - for fixed number of passes
- Non-deterministic Algorithms
 - Simulated Annealing
 - Genetic Algorithms